Nama : Muhammad Dava Desvilano
Nim : 25091397048
Kelas : 2025B

Jelaskan Program Ini

```
Hash Table Premium+ (Linear Probing) — matplotlib + keyboard controls (Windows CMD safe)

Controls:
  SPACE : pause/resume
  RIGHT : step forward (when paused)
  LEFT  : step backward (when paused)
  R     : restart
  ESC/Q : quit

Run:
  python hash_table_premium_keyboard.py
Options:
  python hash_table_premium_keyboard.py --size 12 --nkeys 11
  python hash_table_premium_keyboard.py --save gif
"""

import argparse
import random
import numpy as np

import matplotlib
matplotlib.use("TkAgg")  # CMD Windows GUI backend

import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation


def plan_inserts(keys, size=20):
    table = [None] * size
    frame_data = []
    load_factors = []

    for step, key in enumerate(keys, 1):
        start = hash(key) % size
        idx = start
        probes = 0

        def snap(phase, placed=False):
            frame_data.append({
                "phase": phase,
                "step": step,
                "key": key,
                "start": start,
                "idx": idx,
                "probes": probes,
```

```
            "table": list(table),
            "placed": placed
        })

    snap("start", placed=False)

    while table[idx] is not None:
        snap("collision", placed=False)
        probes += 1
        idx = (idx + 1) % size
        snap("probe", placed=False)

    table[idx] = key
    snap("place", placed=True)

    lf = sum(v is not None for v in table) / size
    load_factors.append(lf)

    snap("pause", placed=True)

return frame_data, load_factors


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--size", type=int, default=20)
    parser.add_argument("--nkeys", type=int, default=18)
    parser.add_argument("--seed", type=int, default=42)
    parser.add_argument("--save", choices=["none", "gif"], default="none")
    args = parser.parse_args()

    random.seed(args.seed)
    keys = [f"k{i}" for i in range(args.nkeys)]
    random.shuffle(keys)

    frames, load_factors = plan_inserts(keys, size=args.size)

    # ==== FIGURE LAYOUT ====
    fig = plt.figure(figsize=(12, 6), constrained_layout=True)
    gs = fig.add_gridspec(2, 1, height_ratios=[3, 1])

    ax_table = fig.add_subplot(gs[0])
    ax_lf = fig.add_subplot(gs[1])

    fig.suptitle("Hash Table Premium+ (Linear Probing) — Keyboard Control", fontsize=14)

    # ---- TABLE GRID ----
    n = args.size
    ax_table.set_xlim(-0.5, n - 0.5)
    ax_table.set_ylim(-0.5, 0.5)
    ax_table.set_yticks([])
```

```
ax_table.set_xticks(range(n))
ax_table.set_xlabel("Index Bucket")

rects, texts = [], []
for i in range(n):
    r = plt.Rectangle((i - 0.5, -0.35), 1.0, 0.7, fill=True, alpha=0.25)
    ax_table.add_patch(r)
    rects.append(r)
    t = ax_table.text(i, 0, "", ha="center", va="center", fontsize=9)
    texts.append(t)

info = ax_table.text(0.01, 1.08, "", transform=ax_table.transAxes, fontsize=11)
legend = ax_table.text(
    0.01, 1.01,
    "SPACE: pause/resume | ←/→: step (pause) | R: restart | Q/Esc: quit",
    transform=ax_table.transAxes, fontsize=9
)

# ---- LOAD FACTOR CHART ----
ax_lf.set_xlim(1, max(2, args.nkeys))
ax_lf.set_ylim(0, 1.05)
ax_lf.set_xlabel("Insert ke-")
ax_lf.set_ylabel("Load Factor")
ax_lf.grid(True, alpha=0.25)

lf_line, = ax_lf.plot([], [])
lf_dot, = ax_lf.plot([], [], marker="o", linestyle="")

bar_bg = plt.Rectangle((0.02, 0.15), 0.96, 0.2, transform=ax_lf.transAxes, alpha=0.15)
bar_fg = plt.Rectangle((0.02, 0.15), 0.00, 0.2, transform=ax_lf.transAxes, alpha=0.35)
ax_lf.add_patch(bar_bg)
ax_lf.add_patch(bar_fg)
lf_text = ax_lf.text(0.02, 0.45, "", transform=ax_lf.transAxes, fontsize=10)

# ==== PLAYER STATE ====
state = {
    "i": 0,         # current frame index
    "paused": False  # paused flag
}

def render(frame):
    """Render a single frame dict."""
    table = frame["table"]
    start = frame["start"]
    idx = frame["idx"]
    phase = frame["phase"]

    # reset buckets
    for j in range(n):
        rects[j].set_alpha(0.25)
        rects[j].set_linewidth(1.0)
```

```
        texts[j].set_text("" if table[j] is None else str(table[j]))

    # highlight target hash bucket
    rects[start].set_alpha(0.55)
    rects[start].set_linewidth(2.5)

    # highlight current idx
    rects[idx].set_alpha(0.75)
    rects[idx].set_linewidth(3.0)

    # collision emphasis
    if phase == "collision":
        rects[idx].set_alpha(0.9)

    filled = sum(v is not None for v in table)
    lf = filled / n

    info.set_text(
        f"Frame {state['i']+1}/{len(frames)} | Step {frame['step']}/{args.nkeys} |
key='{frame['key']}' | "
        f"hash%size={start} | idx={idx} | probes={frame['probes']} | load_factor={lf:.2f} |
phase={phase} | "
        f"{'PAUSED' if state['paused'] else 'PLAY'}"
    )

    # completed inserts so far
    completed = frame["step"] - 1 + (1 if frame["placed"] else 0)
    if completed <= 0:
        x, y = [], []
    else:
        x = list(range(1, completed + 1))
        y = load_factors[:completed]

    lf_line.set_data(x, y)
    if x:
        lf_dot.set_data([x[-1]], [y[-1]])
        bar_fg.set_width(0.96 * y[-1])
        lf_text.set_text(f"Load Factor: {y[-1]:.2f} ({int(y[-1]*100)}%)")
    else:
        lf_dot.set_data([], [])
        bar_fg.set_width(0.00)
        lf_text.set_text("Load Factor: 0.00 (0%)")

    return rects + texts + [info, legend, lf_line, lf_dot, bar_fg, lf_text]

  def update(_):
    """Animation tick: advance if not paused; otherwise keep frame."""
    if not state["paused"]:
        state["i"] = min(state["i"] + 1, len(frames) - 1)
    return render(frames[state["i"]])
```

```
    # Init render
    def init():
        return render(frames[state["i"]])

    ani = FuncAnimation(
        fig, update, init_func=init,
        frames=np.arange(len(frames)),  # timer ticks, actual frame controlled by state["i"]
        interval=450, blit=False, repeat=False
    )

    def on_key(event):
        k = event.key
        if k == " ":
            state["paused"] = not state["paused"]
            fig.canvas.draw_idle()

        elif k == "right":
            if state["paused"]:
                state["i"] = min(state["i"] + 1, len(frames) - 1)
                render(frames[state["i"]])
                fig.canvas.draw_idle()

        elif k == "left":
            if state["paused"]:
                state["i"] = max(state["i"] - 1, 0)
                render(frames[state["i"]])
                fig.canvas.draw_idle()

        elif k in ("r", "R"):
            state["i"] = 0
            state["paused"] = True  # restart dalam kondisi pause biar enak step
            render(frames[state["i"]])
            fig.canvas.draw_idle()

        elif k in ("escape", "q", "Q"):
            plt.close(fig)

    fig.canvas.mpl_connect("key_press_event", on_key)

    if args.save == "gif":
        try:
            from matplotlib.animation import PillowWriter
            ani.save("hash_table_premium_keyboard.gif", writer=PillowWriter(fps=2))
            print("Tersimpan: hash_table_premium_keyboard.gif")
        except Exception as e:
            print("Gagal simpan GIF. Pastikan 'pillow' terpasang. Error:", e)

    plt.show()


if __name__ == "__main__":
```

```
main()
```

Penjelasannya

A. LATAR BELAKANG

Dalam pengaturan data, Hash Table adalah cara penyimpanan data yang memungkinkan pencarian cepat melalui fungsi hash. Meskipun demikian, seringkali muncul masalah yang disebut collision, di mana dua data menghasilkan nilai hash yang identik.

Untuk menangani isu ini, diterapkan metode Linear Probing, yang merupakan teknik untuk mencari slot kosong secara berurutan saat terjadi collision.

Program ini dibuat untuk menyimulasikan dan memvisualisasikan proses tersebut secara interaktif dengan menggunakan Python dan pustaka matplotlib.

B. TUJUAN PROGRAM

Tujuan dari program ini adalah:

1. Menampilkan tahapan memasukkan data ke dalam tabel hash.
2. Memvisualisasikan terjadinya bentrokan.
3. Menjelaskan cara kerja linear probing.
4. Menunjukkan perubahan faktor beban pada setiap penyisipan.
5. Memberikan kontrol yang interaktif menggunakan keyboard.

C. Prinsip Kerja Program

Program bekerja dalam dua tahap utama:

Tahap 1 – Perencanaan Proses Penambahan

Fungsi plan_inserts() memiliki tugas untuk:

- Membentuk tabel yang belum terisi
- Mengacak data (keys)
- Menghitung posisi hash
- Mengatasi tabrakan menggunakan metode probing linier

- Mencatat setiap langkah dalam bentuk frame
- Rumus hash yang diterapkan:

  index = hash(key) % size

  Jika terjadi collision:

  index = (index + 1) % size

Setiap Langkah Seperti

- start
- collision
- probe
- place
- pause

disimpan agar dapat divisualisasikan dalam animasi.

Tahap 2 – Visualisasi

Program menampilkan dua komponen utama:

A. Tampilkan Hash Table

- Setiap bucket ditunjukkan sebagai persegi.
- Ember yang menjadi fokus hash ditandai dengan sorotan.
- Ember yang sedang dianalisis mendapatkan penekanan visual.
- Informasi rinci seperti langkah, kunci, prob, dan faktor muatan ditampilkan.

B. Grafik Load Factor

Load factor dihitung dengan rumus:

load_factor = jumlah_elemen_terisi / ukuran_tabel

Grafik menunjukkan peningkatan load factor setiap insert.

D. Contoh Proses Sederhana

Misalkan ukuran tabel = 4
Data yang dimasukkan = k1, k2

Keadaan awal: [-, -, -, -]

Insert k1
Misal hash menghasilkan index 1: [-, k1, -, -]

Insert k2
Misal hash menghasilkan index 1 (collision)

Karena penuh, maka digeser ke index berikutnya: [-, k1, k2, -]

Load factor setelah dua insert: 2 / 4 = 0.5

E. Sistem Animasi

Program memanfaatkan FuncAnimation dari matplotlib untuk menampilkan setiap bingkai secara bertahap.

Ada variabel yang digunakan untuk mengontrol:

- state["i"] → menunjukkan frame ke berapa
- state["paused"] → menunjukkan status animasi

    - Jika tidak dihentikan:
      Frame akan bertambah secara otomatis.
    - Jika dihentikan:
      Pengguna dapat bergerak maju atau mundur secara manual.

F. Kontrol Interaktif

Program mendukung kontrol keyboard sebagai berikut:

Tombol	**Fungsi**
SPACE	Pause / Resume
Panah Kanan	Maju satu langkah
Panah Kiri	Mundur satu langkah
R	Restart
ESC / Q	Keluar

Fitur ini membuat program bersifat interaktif dan memudahkan analisis setiap langkah.

G. Analisis

Berdasarkan simulasi, dapat diamati bahwa:

- Semakin besar load factor, semakin tinggilah kemungkinan terjadinya tabrakan.
- Probing linier akan membuat pencarian untuk menemukan slot kosong menjadi lebih lama ketika tabel hampir terisi penuh.
- Kinerja hash table sangat bergantung pada ukuran tabel dan volume data.

Jika load factor mendekati 1 (100%), maka waktu pencarian bisa mendekati kompleksitas O(n).

H. Kesimpulan

Program ini sukses memperagakan cara kerja tabel hash menggunakan metode linear probing dengan cara yang visual dan interaktif.

Melalui program ini dapat dipahami bahwa:

- Collision merupakan hal yang umum dalam hash table.
- Linear probing adalah metode sederhana untuk mengatasinya.
- Load factor berperan penting dalam menentukan efisiensi.
- Visualisasi membantu memahami algoritma yang abstrak menjadi lebih konkret.